

Carlos Coronel • Steven Morris

Database Systems

Design, Implementation, & Management

12/19/2023

Database System Design, Implementation And Management



National Economics University
Faculty of Mathematical Economics

Database System Design, Implementation And
Management Course – Data Model

- ❑ Instructor: Duc Minh Vu (MFE)
- ❑ Email: minhvd [at] neu.edu.vn

12/19/2023

Database System Design, Implementation And Management

12

Learning Objectives

1. Discuss data modeling and why data models are important
2. Describe the basic data-modeling building blocks
3. Define what business rules are and how they influence database design
4. Outline how the major data models evolved
5. List emerging alternative data models and the needs they fulfill
6. Explain how data models can be classified by their level of abstraction

Introduction

- ❑ Designers, programmers, and end users see data in different ways
- ❑ Different views of same data lead to designs that do not reflect organization's operation
- ❑ Data modeling reduces complexities of database design
- ❑ Various degrees of data abstraction help reconcile varying views of same data

2-1 Data Modeling and Data Models

- ❑ Data models
 - Relatively simple representations of complex real-world data structures
 - ✓ Often graphical
 - ❑ Model: an abstraction of a real-world object or event
 - Useful in understanding complexities of the real-world environment
- ❑ Data modeling is iterative and progressive

data modeling

The process of creating a specific data model for a determined problem domain.

data model

A representation, usually graphic, of a complex “real-world” data structure. Data models are used in the database design phase of the Database Life Cycle.

2-2 The Importance of Data Models

- ❑ Facilitate interaction among the designer, the applications programmer, and the end user
- ❑ End users have different views and needs for data
- ❑ Data model organizes data for various users
- ❑ Data model is an abstraction
 - Cannot draw required data out of the data model

Note

The terms *data model* and *database model* are often used interchangeably. In this book, the term *database model* is used to refer to the implementation of a *data model* in a specific database system.

Note

An implementation-ready data model should contain at least the following components:

- A description of the data structure that will store the end-user data
- A set of enforceable rules to guarantee the integrity of the data
- A data manipulation methodology to support the real-world data transformations

2-3 Data Model Basic Building Blocks

- ❑ Entity: anything about which data are to be collected and stored
- ❑ Attribute: a characteristic of an entity
- ❑ Relationship: describes an association among entities
 - One-to-many (1:M) relationship
 - Many-to-many (M:N or M:M) relationship
 - One-to-one (1:1) relationship
- ❑ Constraint: a restriction placed on the data

entity

A person, place, thing, concept, or event for which data can be stored. See also *attribute*.

attribute

A characteristic of an entity or object. An attribute has a name and a data type.

relationship

An association between entities.

constraint

A restriction placed on data, usually expressed in the form of rules. For example, "A student's GPA must be between 0.00 and 4.00."

one-to-many (1:M or 1..*) relationship

Associations among two or more entities that are used by data models. In a 1:M relationship, one entity instance is associated with many instances of the related entity.

many-to-many (M:N or *.*) relationship

Association among two or more entities in which one occurrence of an entity is associated with many occurrences of a related entity and one occurrence of the related entity is associated with many occurrences of the first entity.

one-to-one (1:1 or 1..1) relationship

Associations among two or more entities that are used by data models. In a 1:1 relationship, one entity instance is associated with only one instance of the related entity.

2-4 Business Rules

- ☐ Descriptions of policies, procedures, or principles within a specific organization
 - Apply to any organization that stores and uses data to generate information
- ☐ Description of operations to create/enforce actions within an organization's environment
 - Must be in writing and kept up to date
 - Must be easy to understand and widely disseminated
- ☐ Describe characteristics of data as viewed by the company

business rule

A description of a policy, procedure, or principle within an organization. For example, a pilot cannot be on duty for more than 10 hours during a 24-hour period, or a professor may teach up to four classes during a semester.

2-4a Discovering Business Rules

☐ Sources of business rules:

- Company managers
- Policy makers
- Department managers
- Written documentation
 - ✓ Procedures
 - ✓ Standards
 - ✓ Operations manuals
- Direct interviews with end users

2-4a Discovering Business Rules (cont'd.)

- ☐ Standardize company's view of data
- ☐ Communications tool between users and designers
- ☐ Allow designer to understand the nature, role, and scope of data
- ☐ Allow designer to understand business processes
- ☐ Allow designer to develop appropriate relationship participation rules and constraints

2-4b Translating Business Rules into Data Model Components

- ☐ Nouns translate into entities
- ☐ Verbs translate into relationships among entities
- ☐ Relationships are bidirectional
- ☐ Two questions to identify the relationship type:
 - How many instances of B are related to one instance of A?
 - How many instances of A are related to one instance of B?

2-4c Naming Conventions

- ☐ Naming occurs during translation of business rules to data model components
- ☐ Names should make the object unique and distinguishable from other objects
- ☐ Names should also be descriptive of objects in the environment and be familiar to users
- ☐ Proper naming:
 - Facilitates communication between parties
 - Promotes self-documentation

Note

Modern database systems allow the use of table name prefixes for attributes, facilitating naming conventions. For example, CUSTOMER.CREDIT_LIMIT indicates the attribute CREDIT_LIMIT from the CUSTOMER table.

2-5 The Evolution of Data Models

Table 2.1 Evolution of Major Data Models

Generation	Time	Data Model	Examples	Comments
First	1960s–1970s	File system	VMS/VSAM	Used mainly on IBM mainframe systems Managed records, not relationships
Second	1970s	Hierarchical and network	IMS, ADABAS, IDS-II	Early database systems Navigational access
Third	Mid-1970s	Relational	DB2 Oracle MS SQL Server MySQL	Conceptual simplicity Entity relationship (ER) modeling and support for relational data modeling
Fourth	Mid-1980s	Object-oriented Object/relational (O/R)	Versant Objectivity/DB DB2 UDB Oracle	Object/relational supports object data types Star Schema support for data warehousing Web databases become common
Fifth	Mid-1990s	XML Hybrid DBMS	dbXML Tamino DB2 UDB Oracle MS SQL Server PostgreSQL	Unstructured data support O/R model supports XML documents Hybrid DBMS adds object front end to relational databases Support large databases (terabyte size)
Emerging Models: NoSQL	Early 2000s to present	Key-value store Column store	SimpleDB (Amazon) BigTable (Google) Cassandra (Apache) MongoDB Riak	Distributed, highly scalable High performance, fault tolerant Very large storage (petabytes) Suited for sparse data Proprietary application programming interface (API)

2-5a Hierarchical and Network Models

❑ The hierarchical model

- Developed in the 1960s to manage large amounts of data for manufacturing projects
- Basic logical structure is represented by an upside-down “tree”
- Structure contains levels or segments

hierarchical model

An early database model whose basic concepts and characteristics formed the basis for subsequent database development.

This model is based on an upside-down tree structure in which each record is called a segment. The top record is the root segment. Each segment has a 1:M relationship to the segment directly below it.

segment

In the hierarchical data model, the equivalent of a file system's record type.

2-5a Hierarchical and Network Models (cont'd.)

❑ Network model

- Created to represent complex data relationships more effectively than the hierarchical model
- Improves database performance
- Imposes a database standard
- Resembles hierarchical model
- ✓ Record may have more than one parent

network model

An early data model that represented data as a collection of record types in 1:M relationships.

schema

A logical grouping of database objects, such as tables, indexes, views, and queries, that are related to each other.

subscheme

The portion of the database that interacts with application programs.

data manipulation language (DML)

The set of commands that allows an end user to manipulate the data in the database, such as SELECT, INSERT, UPDATE, DELETE, COMMIT, and ROLLBACK.

data definition language (DDL)

The language that allows a database administrator to define the database structure, schema, and subschema.

2-5a Hierarchical and Network Models (cont'd.)

- Collection of records in 1:M relationships
- Set composed of two record types:
 - ✓ Owner
 - ✓ Member

□ Network model concepts still used today:

- Schema
 - ✓ Conceptual organization of entire database as viewed by the database administrator
- Subschema
 - ✓ Database portion “seen” by the application programs

2-5a Hierarchical and Network Models (cont'd.)

- Data management language (DML)
 - ✓ Defines the environment in which data can be managed
- Data definition language (DDL)
 - ✓ Enables the administrator to define the schema components

2-5b The Relational Model

- ❑ Developed by E.F. Codd (IBM) in 1970
- ❑ Table (relations)
 - Matrix consisting of row/column intersections
 - Each row in a relation is called a tuple
- ❑ Relational models were considered impractical in 1970
- ❑ Model was conceptually simple at expense of computer overhead

table (relation)

A logical construct perceived to be a two-dimensional structure composed of intersecting rows (entities) and columns (attributes) that represents an entity set in the relational model.

tuple

In the relational model, a table row.

relational model

Developed by E. F. Codd of IBM in 1970, the relational model is based on mathematical set theory and represents data as independent relations. Each relation (table) is conceptually represented as a two-dimensional structure of intersecting rows and columns. The relations are related to each other through the sharing of common entity characteristics (values in columns).

2-5b The Relational Model (cont'd.)

- ❑ Relational data management system (RDBMS)
 - Performs same functions provided by hierarchical model
 - Hides complexity from the user
- ❑ Relational diagram
 - Representation of entities, attributes, and relationships
- ❑ Relational table stores collection of related entities

relational database management system (RDBMS)

A collection of programs that manages a relational database. The RDBMS software translates a user's logical requests (queries) into commands that physically locate and retrieve the requested data.

relational diagram

A graphical representation of a relational database's entities, the attributes within those entities, and the relationships among the entities.

Figure 2.1 Linking Relational Tables

Table name: AGENT (first six attributes)

AGENT_CODE	AGENT_LNAME	AGENT_FNAME	AGENT_INITIAL	AGENT_AREACODE	AGENT_PHONE
501	Alby	Alex	B	713	228-1249
502	Hahn	Leah	F	615	882-1244
503	Okon	John	T	615	123-5569

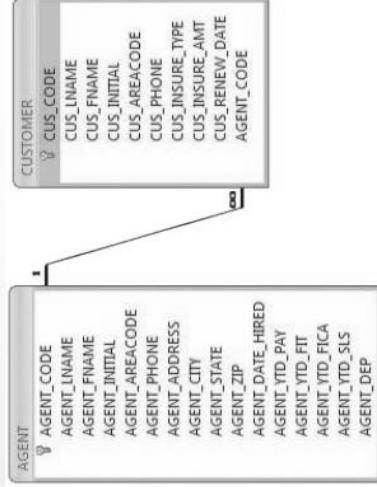
Database name: Ch02_InsureCo

Link through AGENT_CODE

Table name: CUSTOMER

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_INSURE_TYPE	CUS_INSURE_AMT	CUS_RENEW_DATE	AGENT_CODE
10010	Romas	Alfred	A	615	844-2573	T1	100.00	05-Apr-2022	502
10011	Dunne	Leona	K	713	894-1238	T1	250.00	16-Jun-2022	501
10012	Smith	Kathy	W	615	894-2285	S2	150.00	29-Jan-2023	502
10013	Olowski	Paul	F	615	894-2180	S1	300.00	14-Oct-2022	502
10014	Orlando	Myron		615	222-1672	T1	100.00	28-Dec-2023	501
10015	O'Brian	Amy	B	713	442-3381	T2	850.00	22-Sep-2022	503
10016	Brown	James	G	615	297-1228	S1	120.00	25-Mar-2023	502
10017	Williams	George		615	290-2556	S1	250.00	17-Jul-2022	503
10018	Ferriss	Anne	G	713	382-7185	T2	100.00	03-Dec-2022	501
10019	Smith	Olette	K	615	297-3809	S2	500.00	14-Mar-2023	503

Figure 2.2 A Relational Diagram



2-5b The Relational Model (cont'd.)

- ❑ SQL-based relational database application involves three parts:
 - End-user interface
 - ✓ Allows end user to interact with the data
 - Set of tables stored in the database
 - ✓ Each table is independent from another
 - Rows in different tables are related based on common values in common attributes
 - SQL “engine”
 - ✓ Executes all queries

2-5c The Entity Relationship Model

- ❑ Widely accepted standard for data modeling
- ❑ Introduced by Chen in 1976
- ❑ Graphical representation of entities and their relationships in a database structure
- ❑ Entity relationship diagram (ERD)
 - Uses graphic representations to model database components
 - Entity is mapped to a relational table

entity relationship (ER) model (ERM)

A data model that describes relationships (1:1, 1:M, and M:N) among entities at the conceptual level with the help of ER diagrams.

entity relationship diagram (ERD)

A diagram that depicts an entity relationship model's entities, attributes, and relations.

entity instance (entity occurrence)

A row in a relational table.

entity set

A collection of like entities.

2-5c The Entity Relationship Model (cont'd.)

- ❑ Entity instance (or occurrence) is row in table
- ❑ Entity set is collection of like entities
- ❑ Connectivity labels types of relationships
- ❑ Relationships are expressed using Chen notation
 - Relationships are represented by a diamond
 - Relationship name is written inside the diamond
- ❑ Crow's Foot notation used as design standard in this book

connectivity

The type of relationship between entities. Classifications include 1:1, 1:M, and M:N.

Chen notation

See *entity relationship (ER) model*.

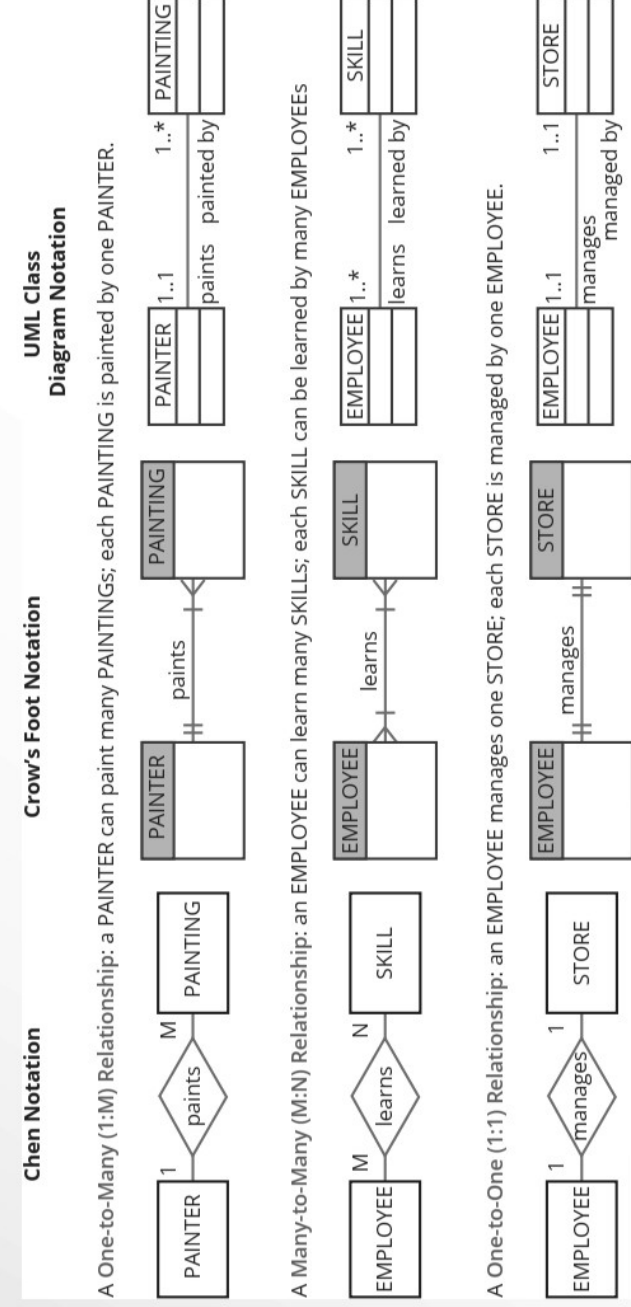
Crow's Foot notation

A representation of the entity relationship diagram that uses a three-pronged symbol to represent the "many" sides of the relationship.

class diagram notation

The set of symbols used in the creation of class diagrams.

Figure 2.3 The ER Model Notations



2-5d The Object-Oriented (OO) Model

- ❑ Data and relationships are contained in a single structure known as an object
- ❑ OODM (object-oriented data model) is the basis for OODBMS
 - Semantic data model
- ❑ An object:
 - Contains operations
 - Are self-contained: a basic building-block for autonomous structures
 - Is an abstraction of a real-world entity

object-oriented data model (OODM)
A data model whose basic modeling structure is an object.

object
An abstract representation of a real-world entity that has a unique identity, embedded properties, and the ability to interact with other objects and itself.

object-oriented database management system (OODBMS)
Data management software used to manage data in an object-oriented database model.

semantic data model
The first of a series of data models that models both data and their relationships in a single structure known as an object.

2-5d The Object-Oriented (OO) Model (cont'd.)

- ❑ Attributes describe the properties of an object
- ❑ Objects that share similar characteristics are grouped in classes
- ❑ Classes are organized in a class hierarchy
- ❑ Inheritance: object inherits methods and attributes of parent class
- ❑ UML based on OO concepts that describe diagrams and symbols
 - Used to graphically model a system

class
A collection of similar objects with shared structure (attributes) and behavior (methods). A class encapsulates an object's data representation and a method's implementation.

method
In the object-oriented data model, a named set of instructions to perform an action. Methods represent real-world actions.

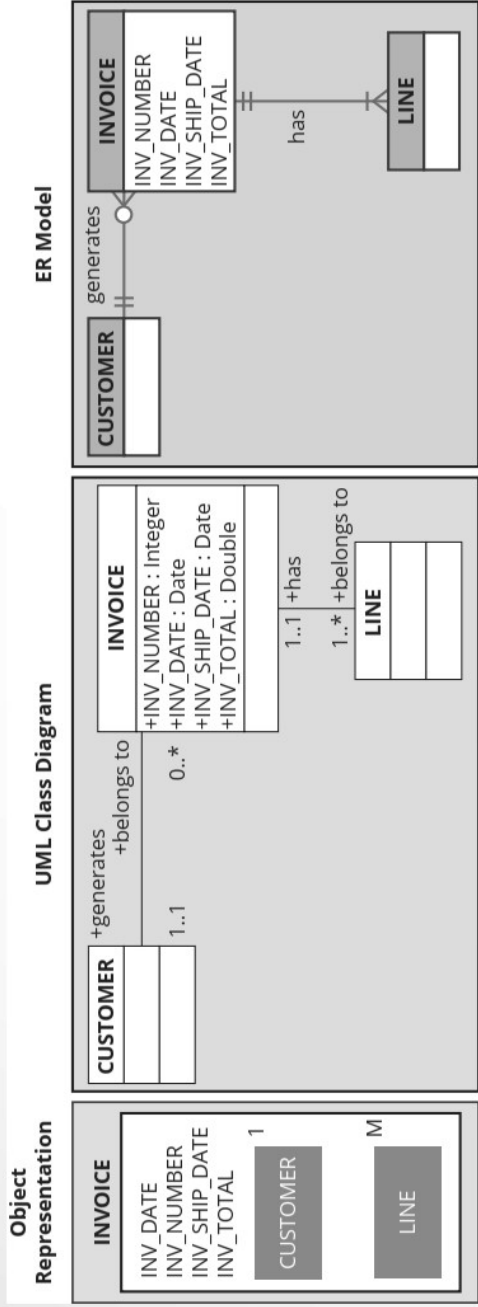
class hierarchy
The organization of classes in a hierarchical tree in which each parent class is a *superclass* and each child class is a *subclass*. See also *inheritance*.

inheritance
In the object-oriented data model, the ability of an object to inherit the data structure and methods of the classes above it in the class hierarchy. See also *class hierarchy*.

Unified Modeling Language (UML)
A language based on object-oriented concepts that provides tools such as diagrams and symbols to graphically model a system.

class diagram
A diagram used to represent data and their relationships in UML object notation.

Figure 2.4 A Comparison of the OO, UML, and ER Models



2-5e Object/Relational and XML

- ❑ Extended relational data model (ERDM)
 - Semantic data model developed in response to increasing complexity of applications
 - Includes many of OO model's best features
 - Often described as an object/relational database management system (O/RDBMS)
 - Primarily geared to business applications

extended relational data model (ERDM)
A model that includes the object-oriented model's best features in an inherently simpler relational database structural environment. See *extended entity relationship model (EERM)*.

object/relational database management system (O/R DBMS)
A DBMS based on the extended relational model (ERDM). The ERDM, championed by many relational database researchers, constitutes the relational model's response to the OODM. This model includes many of the object-oriented model's best features within an inherently simpler relational database structure.

Extensible Markup Language (XML)
A metalanguage used to represent and manipulate data elements. Unlike other markup languages, XML permits the manipulation of a document's data elements. XML facilitates the exchange of structured documents such as orders and invoices over the Internet.

Object/Relational and XML (cont'd.)

- ❑ The Internet revolution created the potential to exchange critical business information
- ❑ In this environment, Extensible Markup Language (XML) emerged as the de facto standard
- ❑ Current databases support XML
 - XML: the standard protocol for data exchange among systems and Internet services

2-5f Emerging Data Models: Big Data and NoSQL

- ❑ Big Data
 - Find new and better ways to manage large amounts of Web-generated data and derive business insight from it
 - Simultaneously provides high performance and scalability at a reasonable cost
 - Relational approach does not always match the needs of organizations with Big Data challenges

Internet of Things (IoT)

A web of Internet-connected devices constantly exchanging and collecting data over the Internet. IoT devices can be remotely managed and configured to collect data and interact with other devices on the Internet.

Big Data

A movement to find new and better ways to manage large amounts of web-generated data and derive business insight from it, while simultaneously providing high performance and scalability at a reasonable cost.

3 Vs

Three basic characteristics of Big Data databases: volume, velocity, and variety.

Hadoop

A Java-based, open-source, high-speed, fault-tolerant distributed storage and computational framework. Hadoop uses low-cost hardware to create clusters of thousands of computer nodes to store and process data.

Hadoop Distributed File System (HDFS)

A highly distributed, fault-tolerant file storage system designed to manage large amounts of data at high speeds.

2-5f Emerging Data Models: Big Data and NoSQL (cont'd.)

❑ NoSQL databases

- Not based on the relational model, hence the name NoSQL
- Supports distributed database architectures
- Provides high scalability, high availability, and fault tolerance
- Supports very large amounts of sparse data
- Geared toward performance rather than transaction consistency

name node

One of three types of nodes used in the Hadoop Distributed File System (HDFS). The name node stores all the metadata about the file system. See also *client node* and *data node*.

data node

One of three types of nodes used in the Hadoop Distributed File System (HDFS). The data node stores fixed-size data blocks (that could be replicated to other data nodes). See also *client node* and *name node*.

client node

One of three types of nodes used in the Hadoop Distributed File System (HDFS). The client node acts as the interface between the user application and the HDFS. See also *name node* and *data node*.

MapReduce
An open-source application programming interface (API) that provides fast data analytics services; one of the main Big Data technologies that allows organizations to process massive data stores.

NoSQL

A new generation of database management systems that is not based on the traditional relational database model.

Emerging Data Models: Big Data and NoSQL (cont'd.)

❑ Key-value data model

- Two data elements: key and value
 - ✓ Every key has a corresponding value or set of values

❑ Sparse data

- Number of attributes is very large
- Number of actual data instances is low

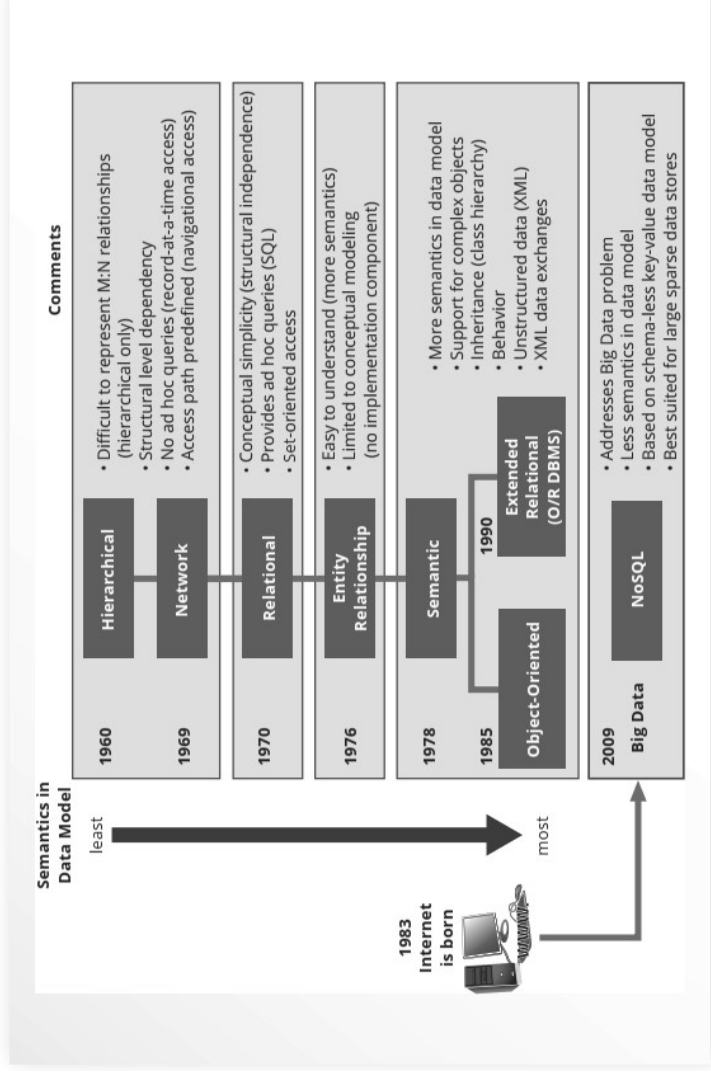
❑ Eventual consistency

- Updates will propagate through system; eventually all data copies will be consistent

2-5g Data Models: A Summary

- ❑ Common characteristics:
 - Conceptual simplicity with semantic completeness
 - Represent the real world as closely as possible
 - Real-world transformations must comply with consistency and integrity characteristics
- ❑ Each new data model capitalized on the shortcomings of previous models
- ❑ Some models better suited for some tasks

Figure 2.5 The Evolution of Data Models



2-6 Degrees of Data Abstraction

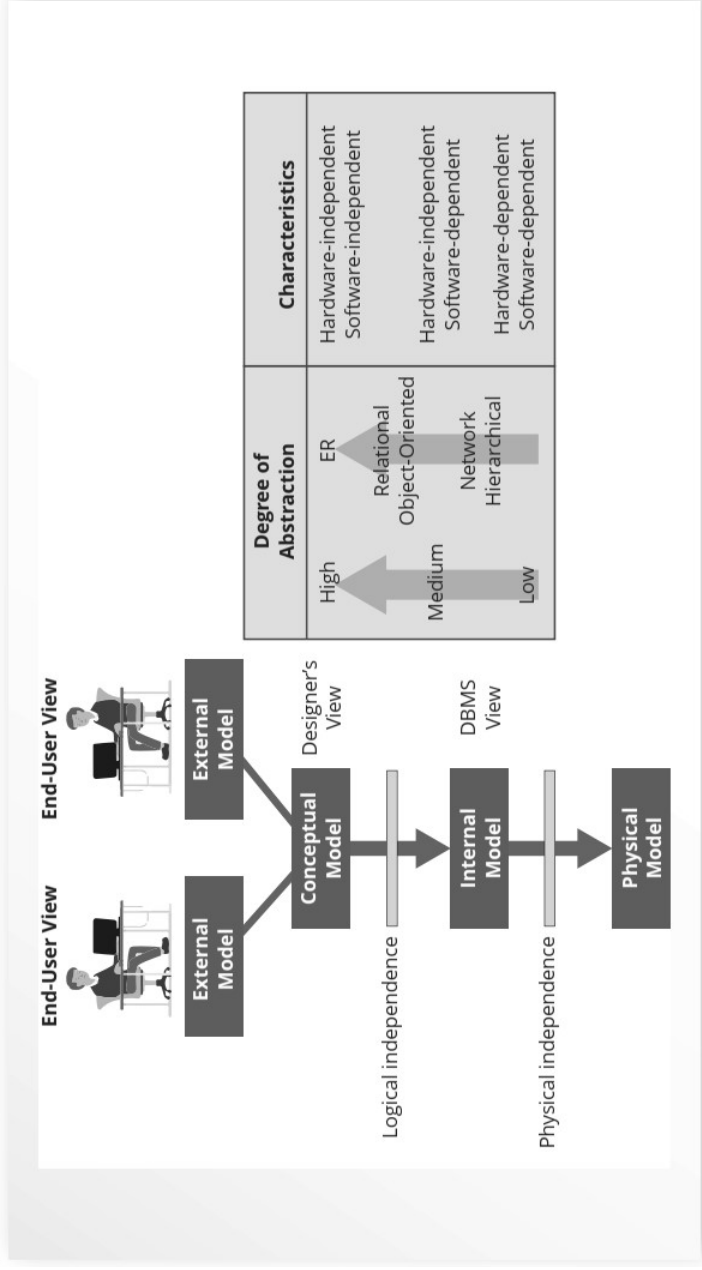
- ❑ Database designer starts with abstracted view, then adds details
- ❑ ANSI Standards Planning and Requirements Committee (SPARC)
 - Defined a framework for data modeling based on degrees of data abstraction (1970s):
 - ✓ External
 - ✓ Conceptual
 - ✓ Internal

Table 2.3 Data Model Basic Terminology Comparison

Real World	Example	File Processing	Hierarchical Model	Network Model	Relational Model	ER Model	OO Model
A group of vendors	Vendor file cabinet	File	Segment type	Record type	Table	Entity set	Class
A single vendor	Global supplies	Record	Segment occurrence	Current record	Row (tuple)	Entity occurrence	Object instance
The contact name	Johnny Ventura	Field	Segment field	Record field	Table attribute	Entity attribute	Object attribute
The vendor identifier	G12987	Index	Sequence field	Record key	Key	Entity identifier	Object identifier

Note: For additional information about the terms used in this table, consult the corresponding chapters and online appendices that accompany this book. For example, if you want to know more about the OO model, refer to Appendix G, Object-Oriented Databases.

Figure 2.6 Data Abstraction Levels



2-6a The External Model

- End users' view of the data environment
- ER diagrams represent external views
- External schema: specific representation of an external view
 - Entities
 - Relationships
 - Processes
 - Constraints

external model

The end user's view of the data environment. Given its business focus, an external model works with a data subset of the global database schema.

external schema

The specific representation of an external view; the end user's view of the data environment.

2-6a The External Model (cont'd.)

- ☐ Easy to identify specific data required to support each business unit's operations
- ☐ Facilitates designer's job by providing feedback about the model's adequacy
- ☐ Ensures security constraints in database design
- ☐ Simplifies application program development

2-6b The Conceptual Model

- ☐ Represents global view of the entire database
- ☐ All external views integrated into single global view: conceptual schema
- ☐ ER model most widely used
- ☐ ERD graphically represents the conceptual schema

conceptual model

The output of the conceptual design process. The conceptual model provides a global view of an entire database and describes the main data objects, avoiding details.

conceptual schema

A representation of the conceptual model, usually expressed graphically. See also *conceptual model*.

software independence

A property of any model or application that does not depend on the software used to implement it.

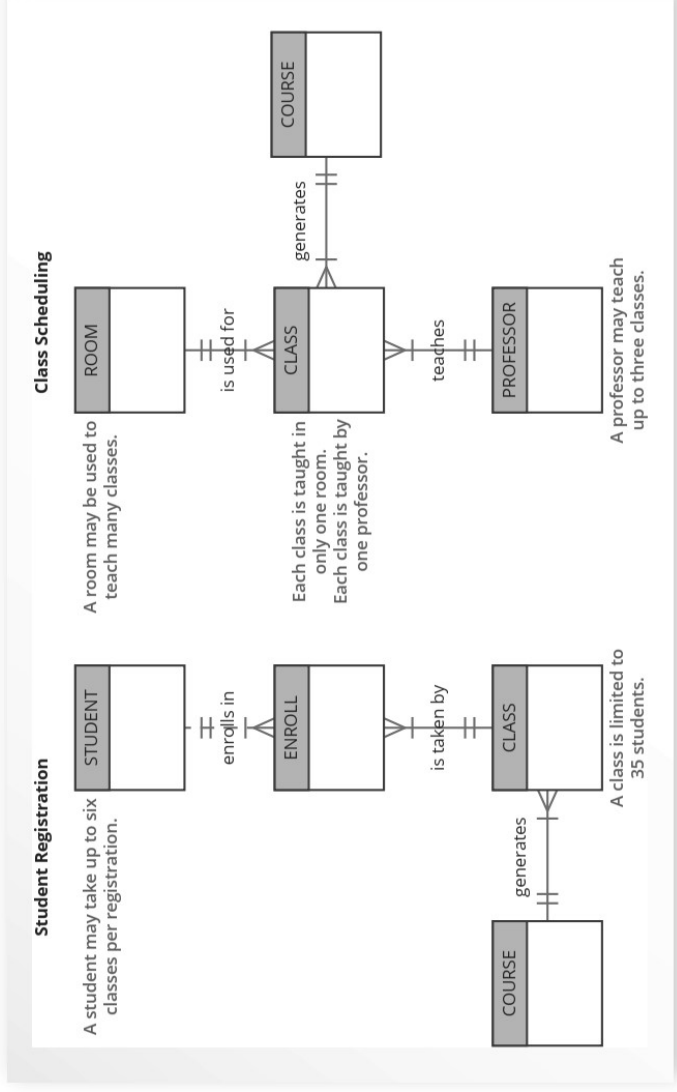
hardware independence

A condition in which a model does not depend on the hardware used in the model's implementation. Therefore, changes in the hardware will have no effect on the database design at the conceptual level.

logical design

A stage in the design phase that matches the conceptual design to the requirements of the selected DBMS and is, therefore, software-dependent. Logical design is used to translate the conceptual design into the internal model for a selected database management system, such as DB2, SQL Server, Oracle, IMS, Informix, Access, or Ingress.

Figure 2.7 External Models for Tiny College



2-6b The Conceptual Model (cont'd.)

- ❑ Provides a relatively easily understood macro level view of data environment
- ❑ Independent of both software and hardware
 - Does not depend on the DBMS software used to implement the model
 - Does not depend on the hardware used in the implementation of the model
 - Changes in hardware or software do not affect database design at the conceptual level

2-6c The Internal Model

- ❑ Representation of the database as “seen” by the DBMS
 - Maps the conceptual model to the DBMS
- ❑ Internal schema depicts a specific representation of an internal model
- ❑ Depends on specific database software
 - Change in DBMS software requires internal model be changed
- ❑ Logical independence: change internal model without affecting conceptual model

internal model

In database modeling, a level of data abstraction that adapts the conceptual model to a specific DBMS model for implementation.

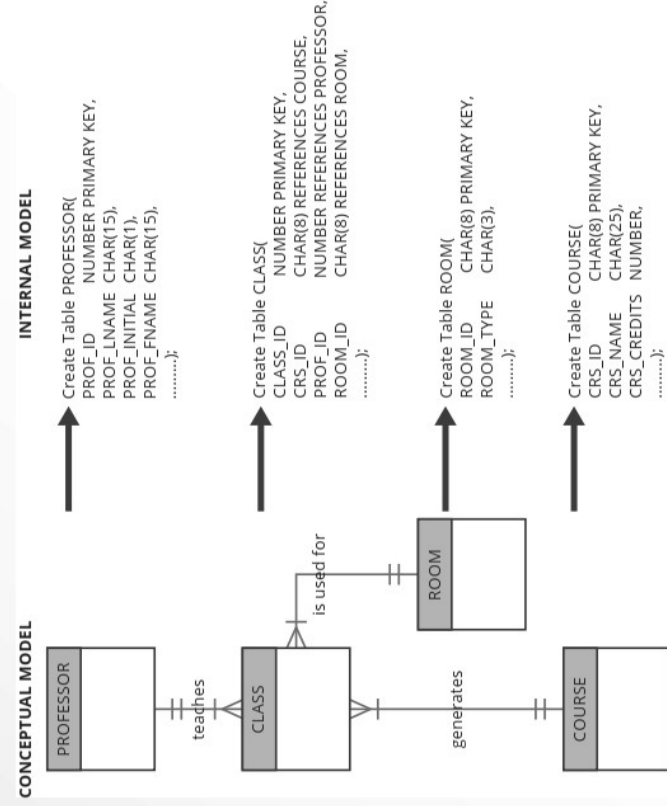
The internal model is the representation of a database as “seen” by the DBMS. In other words, the internal model requires a designer to match the conceptual model’s characteristics and constraints to those of the selected implementation model.

internal schema

A representation of an internal model using the database constructs supported by the chosen database.

logical independence
A condition in which the internal model can be changed without affecting the conceptual model. (The internal model is hardware-independent because it is unaffected by the computer on which the software is installed. Therefore, a change in storage devices or operating systems will not affect the internal model.)

Figure 2.9 Internal Model for Tiny College



2-6d The Physical Model

- ☐ Operates at lowest level of abstraction

➤ Describes the way data are saved on storage media such as disks or tapes

☐ Requires the definition of physical storage and data access methods

☐ Relational model aimed at logical level

➤ Does not require physical-level details

☐ Physical independence: changes in physical model do not affect internal model

physical model

A model in which physical characteristics such as location, path, and format are described for the data. The physical model is both hardware- and software-dependent. See also *physical design*.

physical independence

A condition in which the physical model can be changed without affecting the internal model.

Table 2.4 Levels of Data Abstraction

Model	Degree of abstraction	Focus	Independent of
External	High ↕ Low	End-user views	Hardware and software
Conceptual		Global view of data (database model independent)	Hardware and software
Internal		Specific database model	Hardware
Physical		Storage and access methods	Neither hardware nor software

Summary

- ❑ A data model is an abstraction of a complex real-world data environment
- ❑ Basic data modeling components:
 - Entities
 - Attributes
 - Relationships
 - Constraints
- ❑ Business rules identify and define basic modeling components

Summary (cont'd.)

- ❑ Hierarchical model
 - Set of one-to-many (1:M) relationships between a parent and its children segments
- ❑ Network data model
 - Uses sets to represent 1:M relationships between record types
- ❑ Relational model
 - Current database implementation standard
 - ER model is a tool for data modeling
 - ✓ Complements relational model

Summary (cont'd.)

- ❑ Object-oriented data model: object is basic modeling structure
- ❑ Relational model adopted object-oriented extensions: extended relational data model (ERDM)
- ❑ OO data models depicted using UML
- ❑ Data-modeling requirements are a function of different data views and abstraction levels
 - Three abstraction levels: external, conceptual, and internal

Review Questions

1. Discuss the importance of data models.
2. What is a business rule, and what is its purpose in data modeling?
3. How do you translate business rules into data model components?
4. Describe the basic features of the relational data model and discuss their importance to the end user and the designer.
5. Explain how the entity relationship (ER) model helped produce a more structured relational database design environment.
6. Consider the scenario described by the statement “A customer can make many payments, but each payment is made by only one customer.” Use this scenario as the basis for an entity relationship diagram (ERD) representation.
7. Why is an object said to have greater semantic content than an entity?
8. What is the difference between an object and a class in the object-oriented data model (OODM)?
9. How would you model Question 6 with an OODM? (Use Figure 2.4 as your guide.)
10. What is an ERDM, and what role does it play in the modern (production) database environment?
11. What is a relationship, and what three types of relationships exist?



12. Give an example of each of the three types of relationships.
13. What is a table, and what role does it play in the relational model?
14. What is a relational diagram? Give an example.
15. What is connectivity? (Use a Crow's Foot ERD to illustrate connectivity.)
16. Describe the Big Data phenomenon.
17. What does the term 3 Vs refer to?
18. What is Hadoop, and what are its basic components?
19. What are the basic characteristics of a NoSQL database?
20. Using the example of a medical clinic with patients and tests, provide a simple representation of how to model this example using the relational model.
21. What is logical independence?
22. What is physical independence?